

Polymorphism in C#



Method Overriding

Mr. Bangar Raju



Understanding Method Overriding in C#: A Comprehensive Guide

By Albert Harmon Published July 31, 2017 ⌚ 3 min read Education



TLDR This blog post explains method overriding in C#, detailing its definition, differences from overloading, and practical implementation through examples. It emphasizes the importance of the virtual modifier for allowing method overriding

and clarifies the behavior changes that occur in overridden methods.

Welcome to this tutorial on method overriding in C#. In this post, we will explore what method overriding is, how it differs from method overloading, and how to implement it in your C# applications.

What is Method Overriding?

Method overriding is an object-oriented programming concept that allows a child class to provide a specific implementation of a method that is already defined in its parent class. This is done by reimplementing the parent class's method in the child class with the same method signature (name and parameters).

Example of Method Overriding

Consider the following example:

1. **Parent Class:** ``Class1``
 - Method: ``Test()`` (no parameters)
2. **Child Class:** ``Class2`` inherits from ``Class1``
 - Method: ``Test()`` (same name and parameters)

In this scenario, ``Class2`` overrides the ``Test()`` method of ``Class1``. This means that when ``Test()`` is called on an instance of ``Class2``, the

overridden version in ``class2`` will execute instead of the one in ``Class1``.

Differences Between Overloading and Overriding

It is common to confuse method overloading with method overriding. Here are the key differences:

1. Method Signature

- **Overloading:** Involves defining multiple methods with the same name but different parameters (changing the parameter types or number).
- **Overriding:** Involves redefining a method with the same name and parameters in a child class.

2. Scope of Implementation

- **Overloading:** Can be performed within the same class or between parent and child classes.
- **Overriding:** Can only be performed between parent and child classes.

3. Permission Requirement

- **Overloading:** The child class does not need permission from the parent class to overload a method.
- **Overriding:** The child class requires permission from the parent class, which is granted by declaring the method as ``virtual`` in the parent class.

4. Behavior Definition

- **Overloading:** Allows defining multiple behaviors for a method. For example, a method named ``Show()`` can have different implementations based on the parameters passed.
- **Overriding:** Changes the behavior of the parent class's method in the child class. It does not create multiple behaviors but rather replaces the existing behavior.

Practical Implementation of Method Overriding

To illustrate method overriding, let's create a simple example in C#.

Step 1: Define the Parent Class

```
public class LoadParent
{
    public virtual void Show()
    {
```

```
        Console.WriteLine("Parent's show method is called.");
    }

    public virtual void Test()
    {
        Console.WriteLine("Parent's test method is called.");
    }
}
```

Step 2: Define the Child Class

```
public class LoadChild : LoadParent
{
    public override void Show()
    {
        Console.WriteLine("Child's show method is called.");
    }

    public override void Test()
    {
        Console.WriteLine("Child's test method is called.");
    }
}
```



Step 3: Create Instances and Call Methods

```
class Program
{
    static void Main(string[] args)
    {
        LoadChild child = new LoadChild();
        child.Show(); // Calls child's show method
        child.Test(); // Calls child's test method
    }
}
```

In this example, the `LoadChild` class overrides both the `Show()` and `Test()` methods from the `LoadParent` class. When these methods are called on an instance of `LoadChild`, the overridden methods execute, demonstrating how method overriding works.

Conclusion

Method overriding is a powerful feature in C# that allows for dynamic method resolution and behavior modification in derived classes. By using the `virtual` and `override` keywords, developers can create flexible and maintainable code. Understanding the differences between overloading and overriding is crucial for effective object-oriented programming.

In future tutorials, we will explore more advanced concepts and practical applications of method overriding in C#. Thank you for reading!

